

Student Course Registration System – Final Report

Syed Ahmed Mohiuddin

20 September 2025

Contents

1	Abstract	3
2	Introduction	3
2.1	Background	3
2.2	Problem Statement	3
2.3	Objectives	3
3	System Design & Approach	3
3.1	Architecture	3
3.2	Modules	4
3.3	Technology Stack	4
4	Implementation Details	4
4.1	Authentication	4
4.2	Course Enrollment & Waitlist	4
4.3	Drop Functionality	5
4.4	Configuration Management	5
5	Testing	5
5.1	Unit Testing: JUnit + Mockito.	5
5.2	Test Cases Examples	5
6	Deployment	5
6.1	Local	5
6.2	Cloud Deployment	5
7	Results & Screenshots	6
8	Conclusion & Future Scope	6
8.1	Achievements	6
8.2	Limitations	7
8.3	Future Enhancements	7
9	References	7
A	Functional Requirements	8
B	Non-Functional Requirements	9
C	DynamoDB Schema	10
C.1	Student Table	10
C.2	Course Table	10
C.3	Enrollment Table	10
C.4	StudentLog Table	10
C.5	Relationships between tables:	10

1 Abstract

The Student Course Registration System automates the process of course enrollment for Institute students. It allows students to sign up, log in, view available courses, enroll in courses, manage waitlists, and drop courses while enforcing important constraints like the maximum number of enrolled or waitlisted courses. The system is designed to minimize manual errors, streamline course management, and provide a seamless online experience. The project is implemented using Java, Spring Boot, Thymeleaf, DynamoDB, and is deployed to Cloud Run using GitHub Actions. Unit testing is performed using JUnit and Mockito.

2 Introduction

2.1 Background

Manual course registration often leads to errors such as overbooking, incorrect waitlists, and delays in processing. Automating this process ensures accurate seat allocation, real-time course status updates, and improved student experience.

2.2 Problem Statement

Students face issues with manual registration, including conflicts in course schedules and over-enrollment. There is a need for a robust, online system that manages courses, enrollments, and waitlists efficiently.

2.3 Objectives

- Provide secure student signup and login with session persistence.
- Enable course enrollment with seat and waitlist management and enforce constraints.
- Unit Test the Business Logic.
- Deploy the application on Cloud Run with automated CI/CD using GitHub Actions.

3 System Design & Approach

This section provides a high-level overview of the system's architecture and modules. For a comprehensive, detailed breakdown of the system's specifications, including the functional requirements (FRs) and non-functional requirements (NFRs), please refer to Appendix A and Appendix B. A visual representation of the system's core components can be seen in the conceptual class diagram in Figure 8, and the complete database schema for DynamoDB is detailed in Appendix C.

3.1 Architecture

The system follows a layered architecture:

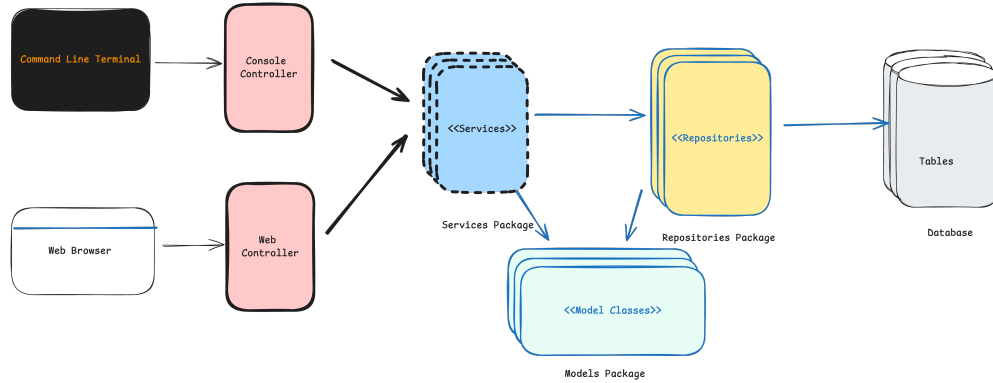


Figure 1: High Level Design Overview

3.2 Modules

- **Authentication:** Signup, login, session management.
- **Course Management:** View courses, check seats, manage waitlists.
- **Enrollment:** Enroll in courses, auto-add to waitlist if full.
- **Drop Courses:** Drop enrolled courses following constraints.

3.3 Technology Stack

- **Backend:** Java + Spring Boot
- **Frontend/UI:** Thymeleaf + HTML/CSS/JS (Material Design login)
- **Database:** AWS DynamoDB
- **Deployment:** Docker + Cloud Run
- **CI/CD:** GitHub Actions (tests + deploy on push)
- **Testing:** JUnit + Mockito

4 Implementation Details

4.1 Authentication

- Session persists across refresh; logout only on manual click.
- Validates email uniqueness and credentials.

4.2 Course Enrollment & Waitlist

- Enroll if seats available; else added to waitlist.
- Enrollments will only happen by LatestEnrollmentBy date.
- Constraints enforced: max 5 enrolled courses, max 3 waitlisted.
- It is assumed that Completed is marked by admin after course ends, out of scope of this project.

4.3 Drop Functionality

- Only enrolled courses can be dropped.
- Students may opt out of waitlist as well.
- Enrollment status will be marked as Withdrawn if students try to Drop/Withdraw before LatestEnrollmentBy date and seat is released.
- When a student opts out of waitlist, or withdraws from a course, if any waitlisted students for that course shall be auto-enrolled till seats are full and waitlist position for rest of the students be adjusted; waitlist management happens asynchronously.
- Drop is not allowed after course End Date.

4.4 Configuration Management

- Students and Courses, Enrollments are stored in JSON files inside resources/ folder, for easy populating the demo data, Data is in sync with business logic.
- Table_INITIALIZER class helps create tables in just single run.

5 Testing

5.1 Unit Testing: JUnit + Mockito.

5.2 Test Cases Examples

- Duplicate email signup → throws exception.
- Invalid login → throws exception.
- Enrolling in 6th course → blocked.
- Adding 4th waitlist course → blocked.
- Dropping course after end date → exception.

6 Deployment

6.1 Local

- Run via Maven commands (`mvn spring-boot:run`).

6.2 Cloud Deployment

- Dockerized application deployed on Cloud Run.
- GitHub Actions pipeline automates test execution and deployment on push.

7 Results & Screenshots

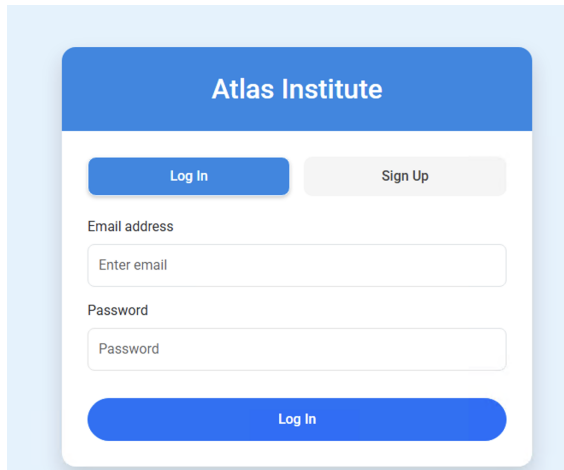


Figure 2: Login Page

All Courses						
Course Name	Status	Start Date	End Date	Seats	Enroll By	Action
AI & Deep Learning	Open	2025-12-10	2026-01-10	Waitlisted	2025-12-01	Apply
Python Basics	Open	2025-11-01	2025-12-01	10 left	2025-10-25	Enroll
Advanced Java	Closed	2025-09-01	2025-12-01	--	2025-08-28	Closed
Databases	Open	2025-12-01	2026-01-01	4 left	2025-11-25	Enroll
Data Structures	Closed	2025-08-01	2025-12-01	--	2025-07-25	Closed
Cloud Computing	Closed	2025-08-20	2025-11-20	--	2025-08-15	Closed
Web Development	Closed	2025-07-01	2025-08-01	--	2025-06-25	Closed
Machine Learning	Closed	2025-09-10	2026-01-10	--	2025-09-05	Closed
Algorithms	Open	2025-10-15	2026-01-15	Waitlisted	2025-10-10	Apply
Java Basics	Closed	2025-01-01	2025-02-01	--	2024-12-25	Closed

Figure 4: Course Listing with Seats/Waitlist

Available Courses				
CourseID	Course Name	Status	Enrollment Ends	Seats
CS110	AI & Deep Learning	Open	2025-12-01	Waitlist
CS103	Python Basics	Open	2025-10-25	☒ 10/10
CS107	Databases	Open	2025-11-25	☒ 4/5
CS105	Algorithms	Open	2025-10-10	Waitlist
CS102	Advanced Java	Closed	2025-08-28	Not Available
CS104	Data Structures	Closed	2025-07-25	Not Available
CS108	Cloud Computing	Closed	2025-08-15	Not Available
CS106	Web Development	Closed	2025-06-25	Not Available
CS109	Machine Learning	Closed	2025-09-05	Not Available
CS101	Java Basics	Closed	2024-12-25	Not Available

Figure 6: Course Listing CLI

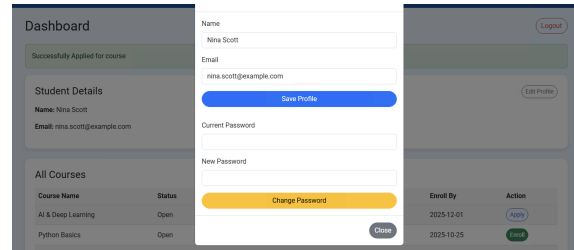


Figure 3: Student Dashboard

My Courses				
Course Name	Start Date	End Date	Status	Action
Algorithms	2025-10-15	2026-01-15	WAITLISTED	Get on
Databases	2025-12-01	2026-01-01	ENROLLED	Withdraw
Cloud Computing	2025-08-20	2025-11-20	DROPPED	

Figure 5: Student Enrollments

Your Courses		
Currently Enrolled 📄		
CourseID	Status	Extra Info
CS107	ENROLLED	
Waitlisted 📄		
CourseID	Status	Extra Info
CS105	WAITLISTED	Position: 4
Past Courses 📄		
CourseID	Status	Extra Info
CS108	DROPPED	

Figure 7: Student Enrollments CLI

8 Conclusion & Future Scope

8.1 Achievements

- Automated course registration system with constraint enforcement.
- Seamless session management and real-time course enrollments.

- Cloud deployment with automated CI/CD.

8.2 Limitations

- No email or SMS notifications.
- Limited reporting features.
- Basic UI design.

8.3 Future Enhancements

- Notification system for enrollment and drop updates.
- Admin dashboard with reports and analytics.

9 References

- Spring Boot Official Documentation: <https://spring.io/projects/spring-boot>
- AWS DynamoDB Docs: <https://docs.aws.amazon.com/dynamodb>
- Docker Docs: <https://docs.docker.com>
- GitHub Actions: <https://docs.github.com/en/actions>
- JUnit & Mockito Docs: <https://junit.org>, <https://site.mockito.org>

A Functional Requirements

FR1 Student Management

- **FR1.1 – Student Sign Up:** A student can create an account with `studentId`, name, email, and password.
- **FR1.2 – Student Login:** A student can log in using `studentId` and password.
- **FR1.3 – Student Profile:** A student can view their own profile, which includes:
 - Basic details (`studentId`, name, email).
 - A list of courses the student has applied for with status: Enrolled, Waitlisted, Completed/Dropped.

FR2 Course Management

- **FR2.1 – Preloaded Courses:** Courses are preloaded in DynamoDB (no admin interface).
- **FR2.2 – Course Attributes:** Each course must have: `courseId`, `courseName`, `maxSeats`, `currentEnrolledCount`, `startDate`, `endDate`, `duration`, `latestEnrollmentBy`.
- **FR2.3 – View Courses:** Students can view all available courses with seat availability.
- **FR2.4 – Enrollment Cutoff:** Students can only enroll if current date $\leq$ `latestEnrollmentBy`.
- **FR2.5 – Drop Cutoff:** Students can only drop if current date $\leq$ `endDate`.

FR3 Enrollment

- **FR3.1 – Course Enrollment:** A student can enroll in a course if: current date $\leq$ `latestEnrollmentBy`, `currentEnrolledCount` $<$ `maxSeats`, student has fewer than 5 active courses.
- **FR3.2 – Course Drop:** A student can drop a course if current date $\leq$ `endDate`. On drop $\rightarrow$ `currentEnrolledCount` $-=$ 1.
- **FR3.3 – Enrollment Count Update:** On successful enroll $\rightarrow$ `currentEnrolledCount` $+=$ 1. On drop $\rightarrow$ `currentEnrolledCount` $-=$ 1.
- **FR3.4 – Prevent Duplicate Enroll:** A student cannot enroll or waitlist twice for the same course.

FR4 Waitlist Management

- **FR4.1 – Waitlist Placement:** If a course is full, the student is placed in a FIFO waitlist queue. A student can be on waitlists for up to 3 courses.
- **FR4.2 – Auto Enrollment from Waitlist:** When a seat opens, check the first student in the waitlist. If their `activeEnrollments` $<$ 5, auto-enroll them. The skipped student remains at the front of the queue.
- **FR4.3 – Simultaneous Seat Openings:** If multiple courses have availability, a student is only auto-enrolled until the max enrollment limit (5) is reached.
- **FR4.4 – Waitlist Status View:** Students can view their waitlist position and can opt-out.

FR5 Seat Availability

- **FR5.1 – Real-time Seat Check:** Every enroll attempt must check seat availability before confirming.
- **FR5.2 – Atomic Updates:** Seat count updates are atomic to prevent overbooking.

FR6 Audit Logging

- **FR6.1 – Action Logging:** Every key action must be recorded in a `StudentLogs` table in DynamoDB.
- **FR6.2 – Log Attributes:** Each log entry must include: `logId`, `studentId`, `action`, `courseId`, `timestamp`, `description`.

B Non-Functional Requirements

NFR1 Usability

- **NFR1.1:** The system must display clear and meaningful messages for all operations.
- **NFR1.2:** Students must be able to view all their enrollments, waitlists, and logs in one place.
- **NFR1.3:** The workflow for sign up → login → enroll/drop must be simple and intuitive.

NFR2 Maintainability & Extensibility

- **NFR2.1:** The system must follow a modular structure (Student Service, Course Service, etc.).
- **NFR2.2:** Future features should be addable without large changes to existing code.
- **NFR2.3:** Business logic must be separated from data access.

NFR3 Audit & Logging

- **NFR3.1:** Every key student action must be logged.
- **NFR3.2:** Logs must include studentId, action, courseId (if applicable), and timestamp.
- **NFR3.3:** Logs must be append-only.

NFR4 Basic Security

- **NFR4.1 – Password Storage:** Passwords must be hashed and salted before storing.
- **NFR4.2 – Session Handling:** A session token must be generated on login and used for subsequent requests.
- **NFR4.3 – Access Control:** A student must only be able to view their own data.
- **NFR4.4 – Input Validation:** The system must validate inputs to prevent invalid data.
- **NFR4.5 – Log Protection:** Audit logs must be append-only.
- **NFR4.6 – Minimal Data Exposure:** Sensitive data like hashed passwords must never be displayed.

NFR5 Basic Reliability

- **NFR5.1:** Seat allocation must be atomic.
- **NFR5.2:** If a course is full, the system must redirect to the waitlist option.
- **NFR5.3:** If a dropped seat is processed, the next eligible waitlist student must be considered.

NFR6 Unit tests written with JUnit & Mockito.

NFR7 DevOps: CI/CD pipeline with GitHub Actions for automated build, test, and deployment.

NFR8 Deployment: Cloud Run with Docker.

C DynamoDB Schema

C.1 Student Table

- **TableName:** Student
- **Primary Key:** `studentId` (PK, String)
- **Attributes:** `studentId` (S), `name` (S), `email` (S, unique), `passwordHash` (S)
- **Global Secondary Indexes (GSI):** **EmailIndex**, Partition key: `email`, Projection: INCLUDE (`studentId`, `name`, `passwordHash`)

C.2 Course Table

- **TableName:** Course
- **Primary Key:** `courseId` (PK, String)
- **Attributes:** `courseId` (S), `courseName` (S), `maxSeats` (N), `seatsFilled` (N), `startDate` (S), `endDate` (S), `latestEnrollmentBy` (S)

C.3 Enrollment Table

- **TableName:** Enrollment
- **Primary Key:** `studentId` (PK, String), `courseId` (SK, String)
- **Attributes:** `studentId` (S), `courseId` (S), `status` (S), `positionWaitlist` (N, nullable), `waitlistedAt` (S), `enrolledAt` (S), `droppedAt` (S), `optedOutAt` (S)
- **Global Secondary Indexes (GSI):** **CourseIndex**, Partition key: `courseId`, Sort key: `studentId`, Projection: ALL

C.4 StudentLog Table

- **TableName:** StudentLog
- **Primary Key:** `logId` (PK, String)
- **Attributes:** `logId` (S), `studentId` (S), `courseId` (S), `action` (S), `timestamp` (S), `description` (S)
- **Global Secondary Indexes (GSI):** **StudentIdIndex**, Partition key: `studentId`, Projection: *KEYS_ONLY*

C.5 Relationships between tables:

- Student (1) ↔ (M) Enrollment (a student can enroll in many courses).
- Course (1) ↔ (M) Enrollment (a course can have many students).
- Enrollment is a junction table between Student and Course.
- StudentLog tracks actions/events by students, often referencing a `courseId`.

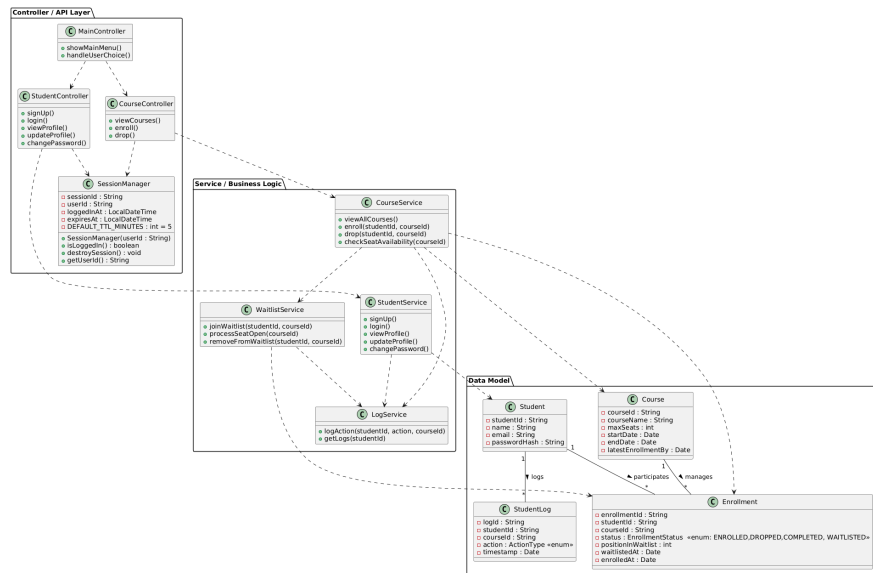


Figure 8: Conceptual Class Diagram of Student Course Registration System